MODUL 01 · FUNDAMENTAL

Pengenalan Flutter, Arsitektur & Setup Lingkungan

Memahami konsep framework Flutter, arsitektur internal (Framework, Engine, Embedder), instalasi Flutter SDK dan Android Studio, mengatasi error setup yang umum, perintah CLI flutter sehari-hari, anatomi proyek, bedah kode counter bawaan baris per baris, perbedaan Hot Reload/Hot Restart/Full Restart, hingga membuat aplikasi pertama 'Halo Kampus'.

Durasi Pertemuan 1 (150 Menit)

Prasyarat Modul 00: Silabus & Persiapan

Capaian Pembelajaran

- Menjelaskan perbedaan arsitektur Native, Hybrid/WebView, dan Flutter Direct Rendering.
- Menginstalasi Flutter SDK, Android SDK, dan ekstensi editor kode, lalu memperbaiki error flutter doctor yang umum.
- Menggunakan perintah CLI flutter: create, devices, emulators, run, pub add, analyze, dan clean.
- Menganalisis peran folder dalam struktur proyek Flutter (android, ios, lib, pubspec.yaml).
- Menjelaskan setiap bagian main.dart bawaan: main, runApp, StatelessWidget, StatefulWidget, setState, dan build.
- Memilih antara Hot Reload, Hot Restart, dan Full Restart sesuai jenis perubahan kode.
- Membuat dan menjalankan aplikasi 'Halo Kampus' pada emulator/perangkat fisik.

Daftar Isi Modul

1. Apa itu Flutter & Mengapa Industri Menggunakannya?
2. Arsitektur Berlapis Flutter (Architecture Layers)
3. Instalasi dan Setup Environment
4. Error Setup yang Sering Muncul & Solusinya
5. Perintah CLI Flutter yang Sering Dipakai
6. Anatomi Struktur Folder Proyek Flutter
7. Bedah main.dart Counter Bawaan Baris per Baris
8. Hot Reload vs Hot Restart vs Full Restart
9. Aplikasi Pertama: Halo Kampus
10. Tugas Praktikum
11. Kuis Pemahaman & Kunci Jawaban

1. Apa itu Flutter & Mengapa Industri Menggunakannya?

Flutter adalah UI Toolkit open-source yang dikembangkan oleh Google untuk membangun aplikasi yang dikompilasi secara native (secara native-compiled) untuk mobile (Android, iOS), web, dan desktop (macOS, Windows, Linux) hanya dari **satu basis kode (single codebase)**.

🚀 Fast Development

Fitur *Stateful Hot Reload* menyuntikkan perubahan kode ke dalam VM Dart dalam hitungan sub-detik tanpa kehilangan status antarmuka aplikasi.

🎨 Expressive & Flexible UI

Flutter mengontrol setiap piksel di layar menggunakan engine render grafis berkinerja tinggi (Skia & Impeller), menjamin tampilan 100% konsisten di semua platform.

⚡ Native Performance

Kode Dart dikompilasi langsung ke kode mesin native ARM (AOT - Ahead-Of-Time compilation) tanpa perantara JavaScript bridge yang lambat.

Perbandingan Pendekatan Pengembangan Mobile

Karakteristik	Native (Kotlin / Swift)	Hybrid (Cordova / Ionic)	Reactive Bridge (React Native)	Flutter (Direct Rendering)
Bahasa	Kotlin / Java (Android), Swift (iOS)	HTML, CSS, JavaScript	JavaScript / TypeScript	**Dart**
Jumlah Codebase	Dua (terpisah Android & iOS)	Satu	Satu (dengan kode native jika perlu)	**Satu (Single Codebase)**
Mekanisme Render	Widget bawaan OS	WebView browser container	Bridge JS memanggil Native Widget	**Render mandiri via Impeller/Skia Engine**
Performa	Sangat Tinggi (100%)	Rendah - Sedang (terbebani WebView)	Tinggi (ada bottleneck bridge)	**Sangat Tinggi (mendekati 60-120 FPS konsisten)**

2. Arsitektur Berlapis Flutter (Architecture Layers)

Flutter dibangun dengan arsitektur modular berlapis-lapis (layered architecture). Ini memberikan abstraksi yang bersih antara logika UI tingkat tinggi dan komunikasi hardware tingkat rendah:



3. Instalasi dan Setup Environment

Berikut adalah langkah-langkah menyiapkan Flutter di komputer pengembangan. Ikuti berurutan dan jangan lanjut ke langkah berikutnya sebelum langkah sebelumnya berhasil.

Langkah 1: Mengunduh Flutter SDK

Unduh paket SDK resmi dari docs.flutter.dev sesuai sistem operasi Anda, lalu ekstrak ke direktori permanen (contoh: `~/development/flutter` di Mac/Linux atau `C:\src\flutter` di Windows).

> ⚠️ **Hindari folder bermasalah**
>
> Jangan ekstrak Flutter ke folder yang butuh hak admin (misal `C:\Program Files\`) atau yang namanya mengandung spasi. Hindari juga folder yang disinkronkan OneDrive/iCloud karena bisa merusak file cache SDK.

Langkah 2: Menambahkan Path ke Environment Variable

macOS / Linux (zsh):

```
# Contoh untuk Mac / Linux (zsh) pada ~/.zshrc:
export PATH="$PATH:$HOME/development/flutter/bin"

# Reload shell configuration
source ~/.zshrc

# Cek: harus menampilkan lokasi folder flutter/bin
which flutter
```

Windows: buka *Start* > ketik "environment variables" > *Edit environment variables for your account* > pilih `Path` > *Edit* > *New* > isi `C:\src\flutter\bin`. Tutup lalu buka ulang terminal, kemudian cek di PowerShell:

```
where.exe flutter
flutter --version
```

Langkah 3: Memasang Android Studio dan Android SDK

Pasang Android Studio, buka sekali agar wizard mengunduh Android SDK. Lalu buka *Settings* > *Languages & Frameworks* > *Android SDK* > tab *SDK Tools* dan pastikan tercentang:

- Android SDK Build-Tools
- Android SDK Command-line Tools (latest)
- Android SDK Platform-Tools
- Android Emulator

Buat emulator lewat *Device Manager* > *Create Virtual Device* (pilih Pixel dengan image Android terbaru).

Langkah 4: Menjalankan flutter doctor

Buka terminal lalu ketik perintah diagnostik utama:

```
flutter doctor

# Versi detail (lokasi SDK, versi Java, dll). Pakai saat mencari penyebab error.
flutter doctor -v
```

Perintah ini akan memindai sistem Anda dan memberi centang hijau jika dependensi sudah terpasang:

```
[✓] Flutter (Channel stable, 3.x.x, on macOS / Windows)
[✓] Android toolchain - develop for Android devices (Android SDK version 3x.x.x)
[✓] Xcode - develop for iOS and macOS (Xcode 16.x)  <-- Opsional untuk pengguna Mac
[✓] Chrome - develop for the web
[✓] Android Studio (version 202x.x)
[✓] VS Code (version 1.9x.x)
[✓] Connected device (2 available)
```

Tanda `[!]` berarti ada peringatan dan `[✗]` berarti komponen belum terpasang. Solusi untuk error yang paling sering muncul dibahas di section berikutnya.

> 💡 **Tips Mahasiswa: Ekstensi VS Code Wajib**
>
> Jika menggunakan Visual Studio Code, buka menu Extensions (`Ctrl+Shift+X` atau `Cmd+Shift+X`) dan pasang:
>
> 1. **Flutter** (oleh Dart-Code) — otomatis memasang dependensi Dart.
> 2. **Awesome Flutter Snippets** — shortcut pengetikan widget cepat (misal ketik `stless` untuk StatelessWidget).
> 3. **Error Lens** — menampilkan error kompilasi langsung di baris kode tanpa perlu membuka tab terminal.

Langkah 5: Menyiapkan HP Android fisik (opsional, disarankan untuk RAM 8 GB)

1. Buka *Setelan* > *Tentang ponsel*, ketuk *Nomor versi / Build number* 7 kali sampai muncul "Anda sekarang developer".
2. Buka *Setelan* > *Opsi pengembang*, aktifkan **USB debugging**.
3. Hubungkan kabel USB (kabel data, bukan kabel charger saja), lalu izinkan dialog "Allow USB debugging" di HP.

4. Cek dari terminal:

```
flutter devices
```

```
Found 3 connected devices:
  SM A155F (mobile) • R58W1234ABC • android-arm64  • Android 14 (API 34)
  macOS (desktop)   • macos       • darwin-arm64   • macOS 15
  Chrome (web)      • chrome      • web-javascript • Google Chrome
```

4. Error Setup yang Sering Muncul & Solusinya

Hampir semua mahasiswa menemui setidaknya satu error saat setup. Jangan panik: baca pesan `flutter doctor`, cari pesan yang sama di tabel ini, lalu jalankan perbaikannya. Setelah memperbaiki, selalu jalankan ulang `flutter doctor`.

1. Android license status unknown

```
[!] Android toolchain - develop for Android devices
    ! Some Android licenses not accepted. To resolve this, run: flutter doctor --android-licenses
```

Penyebab: lisensi Android SDK belum disetujui. **Perbaikan:**

```
flutter doctor --android-licenses
# Ketik y lalu Enter untuk setiap lisensi yang ditanyakan
```

2. cmdline-tools component is missing

```
[✗] Android toolchain - develop for Android devices
    ✗ cmdline-tools component is missing
```

Penyebab: Android SDK Command-line Tools belum dipasang (dan perintah lisensi di atas juga akan gagal karenanya). **Perbaikan:** buka Android Studio > *Settings* > *Languages & Frameworks* > *Android SDK* > tab *SDK Tools* > centang **Android SDK Command-line Tools (latest)** > *Apply*. Jika Flutter tidak menemukan lokasi SDK, beri tahu secara manual:

```
# macOS (lokasi default)
flutter config --android-sdk "$HOME/Library/Android/sdk"

# Windows (lokasi default, ganti NamaUser)
flutter config --android-sdk "C:\Users\NamaUser\AppData\Local\Android\Sdk"
```

3. JAVA_HOME / versi Java tidak cocok

```
ERROR: JAVA_HOME is set to an invalid directory
# atau saat build:
Unsupported class file major version 65
```

Penyebab: Gradle memakai Java yang salah, biasanya JDK lama yang terpasang terpisah. Cara paling aman adalah memakai JDK bawaan Android Studio (folder `jbr`):

```
# Arahkan Flutter ke JDK bawaan Android Studio (macOS)
flutter config --jdk-dir "/Applications/Android Studio.app/Contents/jbr/Contents/Home"

# Atau set JAVA_HOME di ~/.zshrc (macOS), lalu: source ~/.zshrc
export JAVA_HOME="/Applications/Android Studio.app/Contents/jbr/Contents/Home"

# Windows (PowerShell), lalu tutup dan buka ulang terminal
```

```
setx JAVA_HOME "C:\Program Files\Android\Android Studio\jbr"

# Cek hasilnya
flutter doctor -v
```

4. CocoaPods not installed (khusus macOS untuk iOS)

```
[!] Xcode - develop for iOS and macOS
    ✗ CocoaPods not installed.
```

Penyebab: plugin iOS dikelola oleh CocoaPods. **Perbaikan** (pilih salah satu):

```
# Lewat Homebrew (disarankan di Apple Silicon)
brew install cocoapods

# Atau lewat RubyGems
sudo gem install cocoapods

# Jika build iOS gagal setelah menambah package
cd ios
pod repo update
pod install
cd ..
```

Sekalian pastikan Xcode sudah dikonfigurasi:

```
sudo xcode-select --switch /Applications/Xcode.app/Contents/Developer
sudo xcodebuild -runFirstLaunch
```

5. Error lain yang sering muncul

Gejala	Penyebab	Perbaikan
`'flutter' is not recognized` / `command not found: flutter`	PATH belum berisi `flutter/bin`	Ulangi Langkah 2, lalu **buka terminal baru**
`No devices found`	Emulator belum jalan atau USB debugging belum diizinkan	`flutter emulators --launch <id>` atau cabut-colok kabel dan izinkan dialog di HP
Build pertama sangat lama / `Timeout waiting to lock`	Gradle sedang mengunduh dependensi pertama kali	Tunggu dengan internet stabil. Jika macet, jalankan `flutter clean` lalu `flutter run` lagi
`Waiting for another flutter command to release the startup lock`	Ada proses flutter lain yang masih jalan	Tutup terminal/IDE lain. Jika tetap macet, hapus file `flutter/bin/cache/lockfile`
Emulator hitam / sangat lambat	Akselerasi hardware mati atau RAM kurang	Aktifkan virtualisasi (VT-x/AMD-V) di BIOS, atau gunakan HP fisik

> 💡 **Cara bertanya yang baik saat error**
>
> Saat bertanya ke asisten praktikum atau forum, sertakan: (1) hasil lengkap `flutter doctor -v`, (2) perintah yang Anda jalankan, (3) potongan pesan error **paling atas** (bukan hanya baris terakhir). Pesan error yang lengkap biasanya sudah menyebutkan solusinya.

5. Perintah CLI Flutter yang Sering Dipakai

VS Code dan Android Studio punya tombol untuk semua hal, tetapi memahami perintah CLI (terminal) membuat Anda tahu apa yang sebenarnya terjadi dan sangat membantu saat debugging. Berikut perintah yang akan dipakai sepanjang semester.

1. Membuat proyek: `flutter create`

```
# Bentuk paling sederhana (nama proyek: huruf kecil + garis bawah)
flutter create halo_kampus

# Menentukan organisasi (dipakai sebagai package name Android: id.ac.unsoed.halo_kampus)
flutter create --org id.ac.unsoed halo_kampus

# Hanya membuat folder platform tertentu agar proyek lebih ringan
flutter create --platforms=android,ios,web halo_kampus

# Proyek kosong tanpa contoh counter dan komentar panjang
flutter create --empty halo_kampus
```

> ⚠️ **Aturan nama proyek**
>
> Nama proyek harus huruf kecil dan garis bawah (`snake_case`), tidak boleh diawali angka dan tidak boleh memakai tanda minus. `halo-kampus` dan `HaloKampus` akan ditolak, gunakan `halo_kampus` .

2. Melihat dan menyalakan device

```
# Daftar device yang tersambung (HP, emulator yang sudah menyala, Chrome, desktop)
flutter devices

# Daftar emulator yang sudah dibuat di Android Studio
flutter emulators

# Menyalakan emulator berdasarkan id dari perintah di atas
flutter emulators --launch Pixel_8_API_35
```

3. Menjalankan aplikasi: `flutter run`

```
# Jalankan di device default (jika hanya ada satu)
flutter run

# Pilih device tertentu dengan -d (id atau nama dari 'flutter devices')
flutter run -d chrome
flutter run -d emulator-5554
```

```
# Mode release: lebih cepat, tanpa Hot Reload (untuk uji performa)
flutter run --release
```

Selama `flutter run` berjalan, terminal menerima tombol berikut:

Tombol	Fungsi
r	Hot Reload: suntikkan perubahan kode, state dipertahankan
R	Hot Restart: jalankan ulang aplikasi dari `main()`, state direset
h	Tampilkan semua perintah yang tersedia
d	Detach: lepas terminal tanpa menutup aplikasi
c	Bersihkan layar terminal
q	Quit: hentikan aplikasi

4. Mengelola package: `flutter pub`

```
# Menambah package dari pub.dev (otomatis menulis ke pubspec.yaml + mengunduh)
flutter pub add http
flutter pub add provider

# Menambah package khusus pengembangan (masuk ke dev_dependencies)
flutter pub add dev:mocktail

# Menghapus package
flutter pub remove http

# Mengunduh ulang semua package sesuai pubspec.yaml (misal setelah git clone)
flutter pub get

# Melihat package yang punya versi lebih baru
flutter pub outdated
```

5. Memeriksa dan merapikan kode

```
# Cari error dan peringatan tanpa menjalankan aplikasi
flutter analyze

# Rapikan format (indentasi, spasi) seluruh file Dart di folder lib
dart format lib

# Jalankan semua test di folder test/
flutter test
```

6. Membersihkan dan memperbarui

```
# Hapus folder build/ dan cache proyek. Obat pertama untuk error build yang aneh.
flutter clean
flutter pub get

# Perbarui Flutter SDK ke versi stable terbaru
flutter upgrade

# Membuat berkas rilis (dibahas di Modul 16)
flutter build apk --release
```

Ringkasan perintah

Perintah	Kapan dipakai
`flutter doctor`	Cek kesehatan instalasi
`flutter create`	Membuat proyek baru
`flutter devices` / `flutter emulators`	Melihat dan menyalakan device
`flutter run`	Menjalankan aplikasi (debug)
`flutter pub add` / `remove` / `get`	Mengelola package
`flutter analyze`	Cek error sebelum menjalankan / mengumpulkan
`flutter clean`	Saat build error tanpa alasan jelas
`flutter build`	Membuat berkas rilis

6. Anatomi Struktur Folder Proyek Flutter

Untuk membuat proyek baru, jalankan perintah CLI:

```
flutter create halo_kampus
cd halo_kampus
```

Struktur folder yang dihasilkan memiliki fungsi spesifik:

```
halo_kampus/
├── android/          <-- Konfigurasi proyek native Android (Gradle, Manifest, build.gradle)
├── ios/              <-- Konfigurasi proyek native iOS (Podfile, Info.plist, Xcode project)
├── lib/              <-- KODE UTAMA DART KITA (95% waktu coding di sini!)
|   └── main.dart     <-- Titik masuk utama aplikasi (entry point)
├── test/             <-- Berkas unit test dan widget test otomatis
├── web/ & linux/ & windows/ & macos/ <-- Konfigurasi platform desktop & browser
├── pubspec.yaml      <-- MANIFEST PROYEK: Dependensi package, asset gambar, font & versi
├── pubspec.lock      <-- Berkas pengunci versi dependencies riil
└── README.md         <-- Dokumentasi proyek
```

Memahami pubspec.yaml

File `pubspec.yaml` adalah file konfigurasi sentral dalam Flutter (mirip `package.json` di Node.js atau `composer.json` di PHP Laravel):

```yaml
name: halo_kampus
description: "Aplikasi praktikum pertama mahasiswa"
publish_to: 'none'
version: 1.0.0+1

environment:
  sdk: ^3.5.0                # Biarkan sesuai hasil 'flutter create' di laptop Anda

dependencies:
  flutter:
    sdk: flutter
  cupertino_icons: ^1.0.8     # Ikon iOS style
  # Package eksternal ditambahkan lewat perintah 'flutter pub add <nama>', contoh:
  # flutter pub add http

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^6.0.0       # Aturan coding standard resmi (versi mengikuti flutter create)

flutter:
  uses-material-design: true
  # Hapus tanda '#' di dua baris berikut HANYA setelah folder assets/images/ benar-benar dibuat.
  # Jika foldernya belum ada, build akan gagal: "unable to find directory entry in pubspec.yaml".
  # assets:
  #   - assets/images/
```

⚠️ **Hati-Hati dengan Indentasi YAML!**

File `.yaml` **sangat sensitif terhadap spasi**. Gunakan 2 spasi (bukan tab). Salah 1 spasi saja dapat menyebabkan `flutter pub get` gagal membaca file konfigurasi.

▶️ **Cara Menjalankan Contoh Kode di Modul Ini**

- Contoh berlabel `lib/main.dart` adalah **program lengkap**. Hapus seluruh isi `lib/main.dart` bawaan, tempel kode contoh, lalu jalankan `flutter run`.
- Jika aplikasi sudah berjalan lalu Anda mengganti seluruh isi `main.dart`, tekan `R` **(Hot Restart)**, bukan `r` (Hot Reload). Hot Reload tidak menjalankan ulang `main()` dan `initState()`, sehingga tampilan bisa tetap lama atau kosong.
- Contoh berlabel file lain (misal `lib/providers/krs_provider.dart`) dibuat sebagai file baru di folder yang sama persis, karena di-`import` oleh `main.dart`.
- Contoh Dart murni (Modul 02) tidak butuh Flutter: jalankan di dartpad.dev.
- Setelah mengganti `main.dart`, hapus `test/widget_test.dart` bawaan (berisi tes aplikasi counter) agar `flutter test` tidak gagal.

7. Bedah main.dart Counter Bawaan Baris per Baris

Setiap proyek baru dari `flutter create` berisi aplikasi **counter**: layar dengan angka dan tombol +. Aplikasi kecil ini sudah memuat hampir semua konsep dasar Flutter, jadi kita bedah bagian per bagian. Komentar bawaan sudah dihapus dan diganti nomor penanda.

Sebelum itu: aplikasi Flutter paling kecil

Agar terlihat bahwa "semua adalah widget", ini program Flutter terkecil yang bisa dijalankan. Tidak ada AppBar, tema, atau tombol, hanya satu teks di tengah layar:

`lib/main.dart`

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(
    const Center(
      child: Text(
        'Halo, Flutter!',
        textDirection: TextDirection.ltr,
        style: TextStyle(fontSize: 32),
      ),
    ),
  );
}
```



Aplikasi paling kecil: hanya satu Text tanpa MaterialApp

`runApp()` menerima satu widget dan menjadikannya akar (root) seluruh tampilan. Karena tidak memakai `MaterialApp`, kita harus menyebut arah teks (`textDirection`) sendiri. Inilah alasan hampir semua aplikasi dibungkus `MaterialApp`: ia menyediakan arah teks, tema, navigasi, dan banyak hal lain secara otomatis.

Kode counter bawaan

lib/main.dart

```dart
// 1
import 'package:flutter/material.dart';

// 2
void main() {
  runApp(const MyApp());
}

// 3
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    // 4
    return MaterialApp(
      title: 'Flutter Demo',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepPurple),
      ),
      home: const MyHomePage(title: 'Flutter Demo Home Page'),
    );
  }
}

// 5
class MyHomePage extends StatefulWidget {
  const MyHomePage({super.key, required this.title});

  final String title;

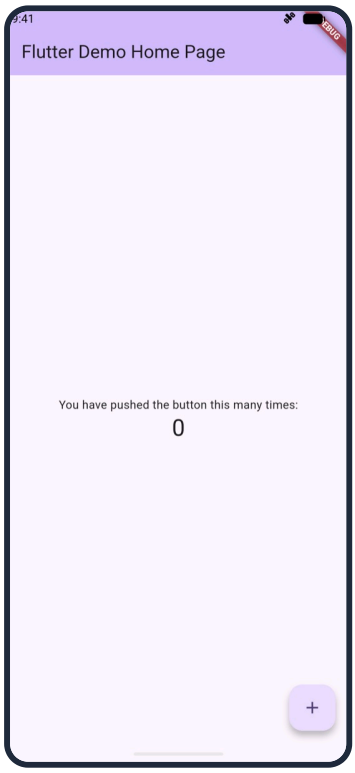
  @override
  State<MyHomePage> createState() => _MyHomePageState();
}

// 6
class _MyHomePageState extends State<MyHomePage> {
  int _counter = 0;

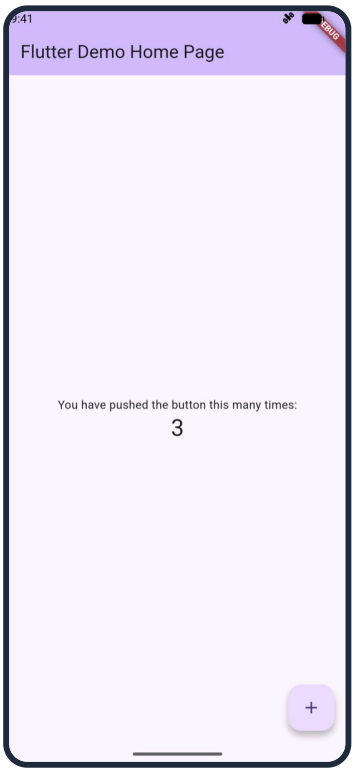
  // 7
  void _incrementCounter() {
    setState(() {
      _counter++;
    });
  }

  // 8
  @override
```

```
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        backgroundColor: Theme.of(context).colorScheme.inversePrimary,
        title: Text(widget.title),
      ),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            const Text('You have pushed the button this many times:'),
            Text(
              '$_counter',
              style: Theme.of(context).textTheme.headlineMedium,
            ),
          ],
        ),
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: _incrementCounter,
        tooltip: 'Increment',
        child: const Icon(Icons.add),
      ),
    );
  }
}
```



Tampilan awal aplikasi counter



Setelah tombol + ditekan tiga kali

ℹ️ **Kode Anda sedikit berbeda?**

Versi Flutter terbaru memakai fitur Dart *dot shorthand*, sehingga `flutter create` menulis `.fromSeed(...)` dan `.center` alih-alih `ColorScheme.fromSeed(...)` dan `MainAxisAlignment.center`. Artinya sama persis. Di materi ini kita memakai bentuk lengkap agar asal setiap nilai terlihat jelas.

Penjelasan baris per baris

No.	Kode	Penjelasan
1	`import 'package:flutter/material.dart';`	Mengimpor pustaka Material Design: semua widget seperti `Scaffold`, `AppBar`, `Text`, dan `Icons` berasal dari sini.
2	`void main()` dan `runApp(const MyApp())`	Titik masuk program. `runApp` memasang widget `MyApp` sebagai akar tampilan. `const` berarti objek dibuat sekali saat kompilasi sehingga lebih hemat.
3	`class MyApp extends StatelessWidget`	Widget akar yang tidak punya data berubah. `const MyApp({super.key})` adalah constructor yang meneruskan `key` ke kelas induk.
4	`MaterialApp(...)`	Pembungkus aplikasi: `title` (nama di task switcher), `theme` (warna dari satu `seedColor` menjadi palet Material 3), dan `home` (halaman pertama).
5	`class MyHomePage extends StatefulWidget`	Halaman yang datanya berubah (angka counter). Field `title` bersifat `final`, diisi lewat parameter `required this.title`. `createState()` membuat objek State pasangannya.
6	`class _MyHomePageState extends State<MyHomePage>`	Tempat data yang bisa berubah disimpan: `int _counter = 0`. Garis bawah (`_`) artinya privat, hanya bisa diakses dari file ini.
7	`setState(() { _counter++; })`	Mengubah data **sekaligus** memberi tahu Flutter agar memanggil `build()` lagi. Tanpa `setState`, angka berubah di memori tetapi layar tidak ikut berubah.
8	`Widget build(BuildContext context)`	Menggambarkan tampilan: `Scaffold` (kerangka halaman) berisi `AppBar`, `body` berupa `Center` > `Column` > dua `Text`, dan `FloatingActionButton` yang memanggil `_incrementCounter` saat ditekan. `widget.title` dipakai untuk membaca field dari kelas `MyHomePage`.

Pohon widget (widget tree) aplikasi counter

```
MyApp
└── MaterialApp
    └── MyHomePage
        └── Scaffold
            ├── AppBar
            |   └── Text (judul)
            ├── Center
            |   └── Column
            |       ├── Text ('You have pushed ...')
            |       └── Text ('$_counter')
            └── FloatingActionButton
                └── Icon (Icons.add)
```

Alur ketika tombol + ditekan

1. Pengguna mengetuk `FloatingActionButton`, sehingga `onPressed` memanggil `_incrementCounter()`.
2. `setState` menjalankan `_counter++` lalu menandai widget sebagai "kotor" (perlu digambar ulang).
3. Pada frame berikutnya Flutter memanggil `build()` lagi, dan `Text('$_counter')` kini berisi angka baru.

> ⚠️ **Kesalahan umum**
>
> - Mengubah `_counter++` di luar `setState`: angka tidak muncul di layar.
> - Menaruh variabel `_counter` di dalam `build()`: nilainya kembali ke 0 setiap kali layar digambar ulang.
> - Menulis `title` alih-alih `widget.title` di dalam kelas State: error `Undefined name 'title'`.

8. Hot Reload vs Hot Restart vs Full Restart

Flutter punya tiga cara menerapkan perubahan kode ke aplikasi yang sedang berjalan. Memilih yang tepat menghemat banyak waktu, dan memilih yang salah membuat Anda mengira kode Anda tidak berfungsi.

Aspek	Hot Reload	Hot Restart	Full Restart (stop + run)
Cara	`r` di terminal, atau simpan file (`Ctrl/Cmd+S`) di VS Code	R di terminal, atau ikon panah melingkar hijau di VS Code	q lalu `flutter run` lagi, atau Stop lalu Run di IDE
Waktu	Kurang dari 1 detik	Beberapa detik	Puluhan detik sampai menit (kompilasi ulang native)
State (data di layar)	Dipertahankan	Direset	Direset
Menjalankan ulang `main()` dan `initState()`	Tidak	Ya	Ya
Kapan dipakai	Mengubah isi `build()`: teks, warna, padding, widget baru	Mengubah `main()`, `initState()`, nilai awal field, variabel global, enum/class	Menambah package berkode native, mengubah file di `android/` atau `ios/`, menambah aset baru

Lab mini: membuktikan perbedaannya

Jalankan kode berikut dengan `flutter run`, lalu tekan tombol di layar sampai angkanya 5. Huruf (A) sampai (E) adalah titik yang akan kita ubah.

`lib/main.dart`

```dart
import 'package:flutter/material.dart';

// Variabel global: nilai awalnya hanya dihitung SEKALI saat pertama dipakai.
final String namaKampus = 'UNSOED';

void main() {
  runApp(const LabReloadApp());
}

class LabReloadApp extends StatelessWidget {
  const LabReloadApp({super.key});
```

```dart
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.indigo), // (A)
      ),
      home: const HalamanLab(),
    );
  }
}

class HalamanLab extends StatefulWidget {
  const HalamanLab({super.key});

  @override
  State<HalamanLab> createState() => _HalamanLabState();
}

class _HalamanLabState extends State<HalamanLab> {
  int _klik = 0; // (B)

  @override
  void initState() {
    super.initState();
    debugPrint('initState dipanggil'); // (C)
  }

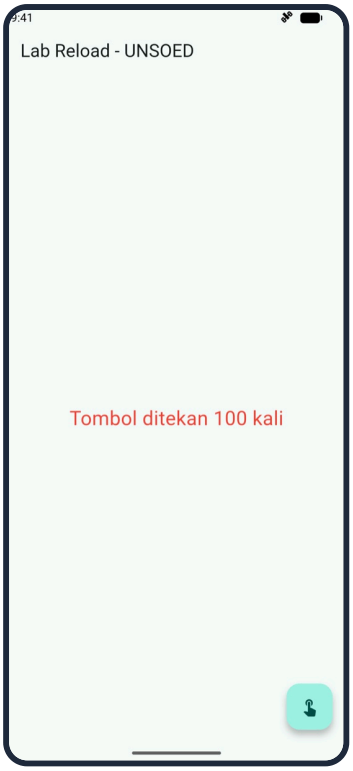
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Lab Reload - $namaKampus')), // (D)
      body: Center(
        child: Text(
          'Tombol ditekan $_klik kali',
          style: const TextStyle(fontSize: 24, color: Colors.black87), // (E)
        ),
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: () => setState(() => _klik++),
        child: const Icon(Icons.touch_app),
      ),
    );
  }
}
```



Kondisi awal percobaan: tombol sudah ditekan 5 kali

Lakukan percobaan berikut satu per satu dan amati hasilnya:

Percobaan	Ubah menjadi	Hot Reload (r)	Hot Restart (R)
(E) warna teks	`Colors.red`	Teks langsung merah, angka tetap 5	Teks merah, angka kembali 0
(A) warna tema	`Colors.teal`	Tombol dan latar halaman berubah nuansa teal, angka tetap 5	Berubah, angka kembali 0
(B) nilai awal	`int _klik = 100;`	**Tidak berubah**, angka tetap 5 karena objek State lama dipertahankan	Angka mulai dari 100
(C) isi `initState`	`debugPrint('initState versi 2');`	Tidak tercetak, `initState` tidak dipanggil lagi	Tercetak di terminal
Variabel global	`final String namaKampus = 'Universitas Jenderal Soedirman';`	Judul AppBar **tetap** "UNSOED"	Judul berubah



Setelah mengubah (A) teal, (B) 100, dan (E) merah lalu menekan Hot Restart: semua perubahan tampil dan angka mulai dari 100

Percobaan (D) menunjukkan sisi sebaliknya: judul AppBar memakai `namaKampus`, tetapi variabel global hanya dihitung sekali. Hot Reload mengganti kode `build()`, bukan nilai yang sudah tersimpan di memori.

Kapan wajib Full Restart?

```
# Contoh: menambah package yang punya kode native (Android/iOS)
flutter pub add shared_preferences

# Hot Reload / Hot Restart TIDAK cukup. Hentikan aplikasi (tekan q), lalu:
flutter run
```

Jika Anda hanya menekan R setelah menambah package native, biasanya muncul error `MissingPluginException`. Ini bukan bug kode Anda; aplikasinya hanya perlu dikompilasi ulang.

> 💡 **Aturan praktis**
>
> Ubah tampilan: r. Ubah data awal atau ganti seluruh `main.dart`: R. Ubah `pubspec.yaml` (package native, aset) atau folder `android/` / `ios/`: stop lalu `flutter run`. Jika ragu dan hasil tampak aneh, naikkan satu tingkat.

9. Aplikasi Pertama: Halo Kampus

Saatnya membuat aplikasi pertama Anda sendiri: **Halo Kampus**, halaman profil mahasiswa dengan tombol presensi. Aplikasi ini memakai konsep yang baru saja dibahas (StatelessWidget, StatefulWidget, `setState`, tema Material 3) ditambah beberapa widget yang akan dipelajari lebih dalam di Modul 03 dan 04.

Langkah 1: Buat proyek

```
flutter create --org id.ac.unsoed halo_kampus
cd halo_kampus
code .            # buka di VS Code (opsional)
```

Langkah 2: Ganti isi `lib/main.dart`

Hapus seluruh isi `lib/main.dart`, lalu ketik (atau tempel) kode berikut:

`lib/main.dart`

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(const HaloKampusApp());
}

/// Akar aplikasi: mengatur tema dan halaman pertama.
class HaloKampusApp extends StatelessWidget {
  const HaloKampusApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Halo Kampus',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: const Color(0xFFF2B705)),
      ),
      home: const ProfilPage(),
    );
  }
}

/// Halaman profil mahasiswa. Stateful karena ada data yang berubah (jumlah hadir).
class ProfilPage extends StatefulWidget {
  const ProfilPage({super.key});

  @override
  State<ProfilPage> createState() => _ProfilPageState();
}

class _ProfilPageState extends State<ProfilPage> {
  // Data mahasiswa: ganti dengan identitas Anda sendiri.
  final String _nama = 'Budi Santoso';
  final String _nim = 'H1D024001';
```

```dart
  final String _prodi = 'S1 Informatika';

  int _jumlahHadir = 0;

  void _presensi() {
    setState(() {
      _jumlahHadir++;
    });
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Presensi ke-$_jumlahHadir tercatat. Semangat!')),
    );
  }

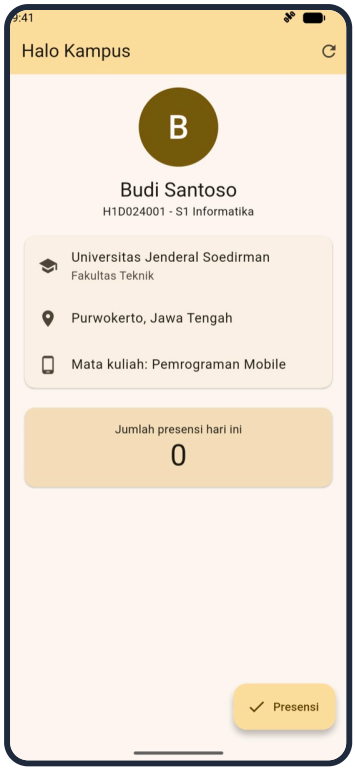
  void _reset() {
    setState(() {
      _jumlahHadir = 0;
    });
  }

  @override
  Widget build(BuildContext context) {
    final warna = Theme.of(context).colorScheme;

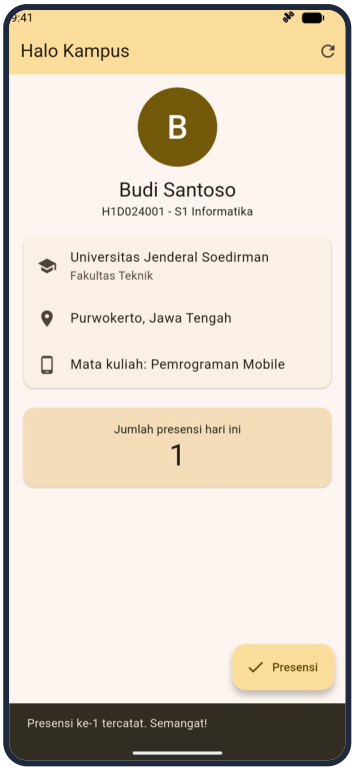
    return Scaffold(
      appBar: AppBar(
        title: const Text('Halo Kampus'),
        backgroundColor: warna.primaryContainer,
        actions: [
          IconButton(
            tooltip: 'Reset presensi',
            onPressed: _reset,
            icon: const Icon(Icons.refresh),
          ),
        ],
      ),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          // Bagian 1: foto dan nama
          CircleAvatar(
            radius: 48,
            backgroundColor: warna.primary,
            child: Text(
              _nama[0],
              style: TextStyle(fontSize: 40, color: warna.onPrimary),
            ),
          ),
          const SizedBox(height: 12),
          Text(
            _nama,
            textAlign: TextAlign.center,
            style: Theme.of(context).textTheme.headlineSmall,
          ),
          Text(
            '$_nim - $_prodi',
            textAlign: TextAlign.center,
            style: Theme.of(context).textTheme.bodyMedium,
          ),
          const SizedBox(height: 16),
```

```
          // Bagian 2: kartu informasi
          Card(
            child: Column(
              children: const [
                ListTile(
                  leading: Icon(Icons.school),
                  title: Text('Universitas Jenderal Soedirman'),
                  subtitle: Text('Fakultas Teknik'),
                ),
                ListTile(
                  leading: Icon(Icons.location_on),
                  title: Text('Purwokerto, Jawa Tengah'),
                ),
                ListTile(
                  leading: Icon(Icons.phone_android),
                  title: Text('Mata kuliah: Pemrograman Mobile'),
                ),
              ],
            ),
          ),
          const SizedBox(height: 16),

          // Bagian 3: penghitung presensi
          Card(
            color: warna.secondaryContainer,
            child: Padding(
              padding: const EdgeInsets.all(16),
              child: Column(
                children: [
                  const Text('Jumlah presensi hari ini'),
                  Text(
                    '$_jumlahHadir',
                    style: Theme.of(context).textTheme.displaySmall,
                  ),
                ],
              ),
            ),
          ),
        ],
      ),
      floatingActionButton: FloatingActionButton.extended(
        onPressed: _presensi,
        icon: const Icon(Icons.check),
        label: const Text('Presensi'),
      ),
    );
  }
}
```



Tampilan awal Halo Kampus



Setelah tombol Presensi ditekan: angka naik menjadi 1 dan SnackBar muncul di bawah

Langkah 3: Hapus test bawaan dan jalankan

```
# test/widget_test.dart menguji aplikasi counter yang sudah tidak ada
rm test/widget_test.dart        # Windows PowerShell: Remove-Item test\widget_test.dart

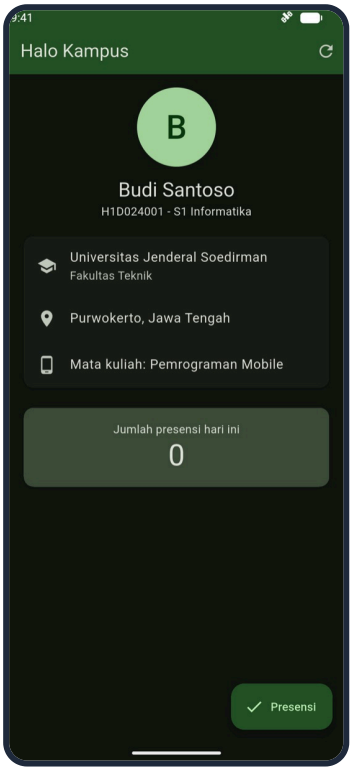
flutter analyze                 # pastikan: No issues found!
flutter run
```

Penjelasan bagian penting

- `HaloKampusApp` hanya mengatur tema dan halaman awal, sehingga cukup Stateless. `seedColor: Color(0xFFF2B705)` adalah kuning keemasan; Material 3 otomatis membuat palet lengkap dari satu warna ini. Format `0xFF` + kode hex RGB: `FF` berarti tidak transparan.
- `_ProfilPageState` menyimpan data mahasiswa sebagai `final` (tidak berubah) dan `_jumlahHadir` sebagai data yang berubah.
- `_presensi()` menaikkan angka dengan `setState`, lalu menampilkan `SnackBar` (pesan singkat di bawah layar) lewat `ScaffoldMessenger`.
- `Theme.of(context).colorScheme` mengambil warna dari tema, sehingga mengganti `seedColor` otomatis mengubah warna AppBar, avatar, dan kartu.
- `ListView` dipakai sebagai body agar halaman bisa di-scroll di layar kecil, tidak muncul error overflow garis kuning-hitam.
- `_nama[0]` mengambil huruf pertama nama untuk ditampilkan di avatar.

Langkah 4: Eksperimen

1. Ganti `_nama`, `_nim`, dan `_prodi` dengan data Anda, lalu tekan R (field ini adalah nilai awal State, ingat percobaan di section sebelumnya).
2. Ganti `seedColor` menjadi `Colors.green` lalu tekan `r`. Tekan Presensi beberapa kali dulu agar terlihat bahwa angkanya tidak hilang.
3. Tambahkan satu `ListTile` baru berisi hobi Anda, misalnya dengan ikon `Icons.sports_soccer`.
4. Ubah tema menjadi gelap dengan menambahkan `brightness: Brightness.dark` di dalam `ColorScheme.fromSeed(...)`.



Hasil eksperimen 2 dan 4: seedColor Colors.green dengan brightness: Brightness.dark

⚠️ **Kesalahan umum**

- **Error `Undefined name '_nama'`**: field ditulis di kelas `ProfilPage`, bukan di `_ProfilPageState`.
- **Menambah `ListTile` tidak muncul / error const**: daftar `children: const [...]` hanya boleh berisi widget const. Pastikan widget baru juga const (semua argumennya nilai tetap).
- **Kurung tutup kurang**: aktifkan *Format on Save* di VS Code dan perhatikan koma di akhir argumen (*trailing comma*) agar formatter menyusun kode bertingkat dengan rapi.

Tugas Praktikum: Praktikum 1: Membuat & Menjalankan Proyek 'Halo Kampus'

Sasaran Praktikum

Membuktikan kesiapan environment, memahami alur kerja CLI dan Hot Reload/Hot Restart, serta memodifikasi aplikasi perdana di emulator atau smartphone.

Langkah Pengerjaan

1. Jalankan `flutter doctor -v`. Perbaiki semua tanda `[✗]` pada Flutter, Android toolchain, dan Connected device (lihat section 4). Simpan hasil akhirnya sebagai `doctor.txt`.
2. Buat proyek baru dengan perintah: `flutter create --org id.ac.unsoed praktikum1_kampus`, lalu buka folder proyek di VS Code.
3. Cek device dengan `flutter devices`. Jika memakai emulator, nyalakan lewat `flutter emulators --launch <id>`.
4. Jalankan aplikasi counter bawaan dengan `flutter run`. Di file `CATATAN.md`, tulis penjelasan singkat dengan kata-kata sendiri untuk bagian bernomor 1 sampai 8 dari section 7.
5. Ganti seluruh isi `lib/main.dart` dengan kode Halo Kampus (section 9), hapus `test/widget_test.dart`, lalu ubah nama, NIM, dan prodi menjadi milik Anda. Ubah judul AppBar menjadi `'Portal Mahasiswa – [NIM Anda]'`.
6. Tambahkan minimal dua `ListTile` baru (misal hobi dan angkatan) dan ganti `seedColor` dengan warna pilihan Anda.
7. Lakukan percobaan Hot Reload vs Hot Restart: tekan Presensi sampai 5, ubah warna (tekan r), lalu ubah nilai awal `_jumlahHadir` menjadi 10 (tekan r, kemudian R). Catat hasil tiap langkah di `CATATAN.md` beserta alasannya.
8. Jalankan `flutter analyze` dan pastikan hasilnya *No issues found!*.
9. Ambil tangkapan layar aplikasi yang berjalan (sebelum dan sesudah percobaan Hot Restart).
10. Salin proyek ke folder `praktikum-01/` di repositori GitHub Praktikum 0, sertakan `doctor.txt`, `CATATAN.md`, dan tangkapan layar, lalu commit dan push.

Output yang Diharapkan

Aplikasi Halo Kampus berjalan dengan identitas mahasiswa, NIM di AppBar, minimal dua ListTile tambahan dan warna tema sendiri; repositori berisi doctor.txt, CATATAN.md (penjelasan main.dart dan hasil percobaan reload), serta tangkapan layar.

Rubrik Penilaian

Aplikasi berjalan tanpa error dan flutter analyze bersih (35%), Modifikasi identitas, ListTile, dan tema (20%), Ketepatan catatan percobaan Hot Reload/Hot Restart (25%), Penjelasan main.dart dengan kata-kata sendiri (10%), Kelengkapan berkas di GitHub (10%).

Kuis Pemahaman

1. Mengapa Flutter memiliki performa render grafis yang mendekati aplikasi native murni?

A. Karena Flutter menggunakan WebView berkecepatan tinggi

B. Karena Flutter memiliki Engine grafis sendiri (Skia/Impeller) yang menggambar langsung ke layar tanpa jembatan (bridge) JavaScript

C. Karena semua kode Flutter diubah menjadi file HTML5 sebelum dijalankan

D. Karena Flutter tidak mendukung animasi kompleks

2. File manakah dalam proyek Flutter yang berfungsi untuk mendaftarkan dependensi pihak ketiga (library dari pub.dev) serta aset gambar?

A. lib/main.dart

B. android/build.gradle

C. pubspec.yaml

D. pubspec.lock

3. Fitur Flutter apakah yang memungkinkan developer melihat perubahan kode di antarmuka dalam waktu kurang dari 1 detik tanpa me-restart aplikasi?

A. Cold Restart

B. Ahead-of-Time Compiling

C. Stateful Hot Reload

D. Garbage Collector

4. Anda mengubah `int _counter = 0;` menjadi `int _counter = 50;` lalu menekan Hot Reload, tetapi angka di layar tidak berubah. Apa yang seharusnya dilakukan?

A. Menjalankan flutter clean

B. Menekan Hot Restart (R) karena nilai awal field State hanya dibaca saat objek State dibuat

C. Menginstal ulang Flutter SDK

D. Menambahkan package baru di pubspec.yaml

5. Apa yang terjadi jika `_counter++` dijalankan di luar `setState()` pada aplikasi counter?

A. Aplikasi langsung crash

B. Nilai variabel bertambah di memori, tetapi layar tidak digambar ulang sehingga angka tidak berubah

C. Flutter otomatis memanggil setState

D. Nilai _counter kembali ke 0

6. flutter doctor menampilkan pesan `cmdline-tools component is missing`. Langkah perbaikan yang tepat adalah...

A. Memasang Android SDK Command-line Tools (latest) melalui SDK Manager di Android Studio

B. Menjalankan flutter upgrade berulang kali

C. Menghapus folder lib/ dari proyek

D. Memasang CocoaPods

7. Perintah manakah yang tepat untuk menambahkan package http ke proyek Flutter?

A. flutter install http

B. npm install http

C. flutter pub add http

D. flutter create http

8. Setelah menambah package yang memiliki kode native (misalnya shared_preferences), muncul MissingPluginException saat Hot Restart. Apa penyebabnya?

A. Kode Dart salah ketik

B. Package tersebut tidak mendukung Android

C. Bagian native aplikasi belum dikompilasi ulang, sehingga perlu stop lalu flutter run lagi (Full Restart)

D. Emulator kehabisan baterai

Kunci Jawaban & Pembahasan

1. B — Flutter tidak menggunakan web view atau jembatan komunikasi JS bridge saat runtime. Engine grafisnya (Skia/Impeller) mengontrol setiap piksel dan berkomunikasi langsung dengan GPU perangkat.

2. C — File pubspec.yaml adalah file manifest utama tempat developer mendeklarasikan library/package eksternal, aset gambar, icon font, dan metadata proyek.

3. C — Stateful Hot Reload menyuntikkan kode baru ke dalam Dart Virtual Machine yang sedang berjalan tanpa menghapus state saat ini, memungkinkan iterasi UI super cepat.

4. B — Hot Reload mempertahankan objek State yang sudah ada, sehingga nilai awal field tidak dihitung ulang. Hot Restart membuat ulang seluruh aplikasi dari main(), sehingga nilai awal baru dipakai.

5. B — setState() memberi tahu framework bahwa ada data yang berubah sehingga build() dipanggil lagi. Tanpa itu, data berubah tetapi tampilan tidak ikut diperbarui.

6. A — cmdline-tools adalah komponen Android SDK yang dipasang lewat Android Studio > Settings > Android SDK > SDK Tools. Setelah terpasang, jalankan flutter doctor --android-licenses.

7. C — flutter pub add menulis package ke pubspec.yaml dengan versi terbaru yang kompatibel lalu langsung mengunduhnya.

8. C — Hot Reload dan Hot Restart hanya memperbarui kode Dart. Plugin native harus ikut dikompilasi ke aplikasi, sehingga aplikasi perlu dihentikan dan dijalankan ulang dengan flutter run.